Future System Architecture

Science Adventure → Android App + Web App + Backend Server + Central Database with Email Authentication and Offline-First Data Synchronisation

Status: DESIGN AND PLAN ONLY. Nothing in this document has been built.

Based on: audit of commit `9fbdaf4` (2026-10-04); see Software_Technical_Audit_Report.pdf

Recommendation: B — Refactor the existing Flutter application; add NestJS + PostgreSQL; Flutter Web client

Contents

1. Current System Summary
2. Problems Blocking Expansion
3. Target Architecture
4. Recommended Technology Stack
5. Backend Architecture
6. Database Architecture
7. Android Architecture
8. Web Architecture
9. Authentication Architecture
10. Authorization Model
11. API Architecture
12. Synchronization Architecture
13. Offline Strategy
14. Conflict Resolution
15. Deployment Architecture
16. Backup Strategy
17. Monitoring Strategy
18. Migration Strategy
19. Development Roadmap
20. Risks
21. Final Recommendation
22. Appendix: Environments, Future Repository Structure

1. Current System Summary

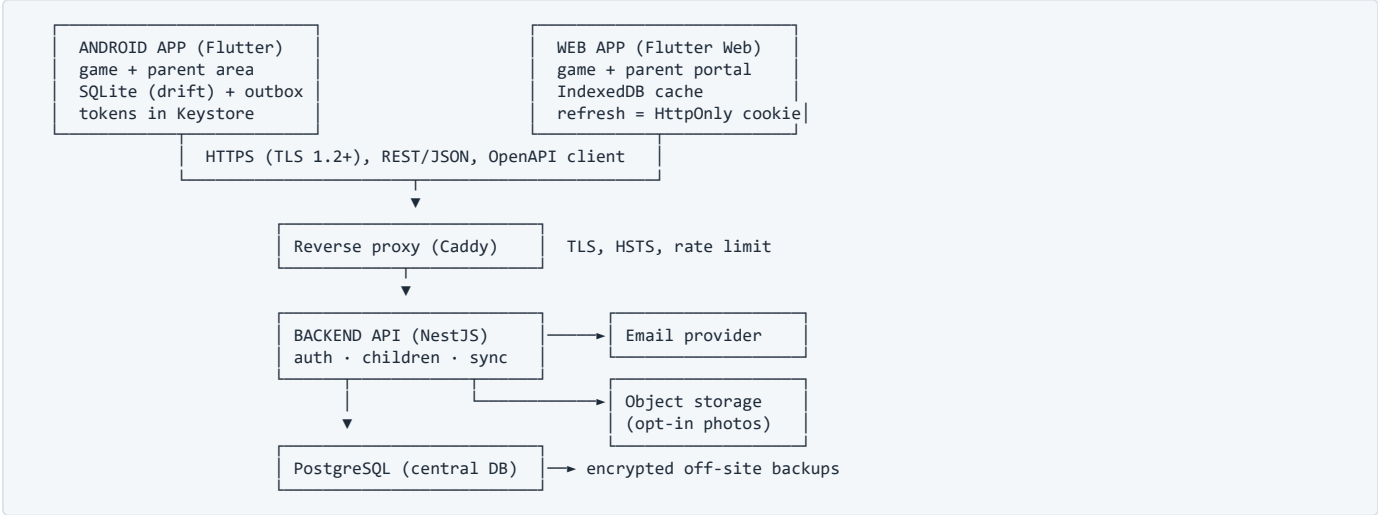
An offline Flutter 3.47.5 game (≈22.4k Dart LOC) for Android. It uses Riverpod state, Flame scenes and bundled JSON lessons. All data lives in one Hive key containing a JSON blob: `{schemaVersion:3, activeChildId, children:{id → {profile, settings, dailySeconds, subjects.science{attempts, journeys}}}}`. There are no accounts, no network and no permissions. Multiple child profiles share a device behind a parent gate.

What the user wants to move to a server (the "data" in this product): parent account, child profiles, per-child settings, assessment attempts (scores), in-progress lesson state ("journeys"), and usage time. Lesson content stays bundled in the apps and is versioned by `contentVersion`.

2. Problems Blocking Expansion

Blocker	Why it blocks sync / web	Ref
Single JSON blob persistence	No per-record versioning, change tracking or partial updates; an outbox is impossible	AUD-004
Untyped `Map<String,dynamic>` state	No schema shared with a server; validation impossible	AUD-003
Migrations inside widgets	The server would need to replicate per-unit migration code	AUD-007
Non-global IDs	Default child id `'local-child'`; attempt IDs `'{lesson}-{micros}'` are not globally unique	models.dart, session.dart
Web bug in ID generation	Deleting the active child fails on web	AUD-005
No logging or CI	Distributed systems need observability and automated tests	AUD-001/009
Product decision "no accounts / no data sending"	Must be formally revised, with privacy and legal input (children's data)	REC-01

3. Target Architecture



Target user journey. A parent creates an account by email on Android, logs in and links the local children. Play results are saved locally and synced. Later the parent opens the web app on a computer, logs in with the same email, sees the same children and progress, and edits a child's settings. Android pulls the change on its next sync (on resume, within 15 minutes, or via "Sync now").

4. Recommended Technology Stack

Layer	Choice	Why (grounded in this project)	Alternatives evaluated
Mobile	**Flutter (keep)** + drift (SQLite) + flutter_secure_storage + dio/http client generated from OpenAPI	22k LOC of working UI and Flame scenes; tests; offline-first needs a relational local DB	Native Kotlin rewrite: no benefit, loses the web reuse
Web	**Flutter Web** (same codebase)	The app already compiles for web (conditional imports, working preview, CanvasKit bundled). The game is canvas/Flame-heavy, so a React/Vue/Angular port would mean rewriting every scene.	Next.js: best for SEO and a marketing/legal site. React/Vue/Angular: would duplicate the game. Recommended only for a separate public site.
Backend	**NestJS (TypeScript, Node 22 LTS)** as a modular monolith	Opinionated structure (modules, controllers, services, guards, pipes) maps directly to the required design; first-class OpenAPI generation → typed Dart client; large talent pool; good fit for I/O-bound JSON sync	.NET / Spring Boot: excellent but heavier for a small team. Django / FastAPI: good, and the team already uses Python for content tooling; viable second choice. **Serverpod (Dart)**: shares models with Flutter end to end; smaller ecosystem. **Supabase**: fastest path (managed Postgres + Auth + RLS), less control over sync semantics.
Database	**PostgreSQL 17/18**	Relational integrity for ownership (FKs), `citext` emails, JSONB for versioned journey state, row-level locking for optimistic concurrency, native `uuid` (PG 18 has built-in `uuidv7()`)	MySQL: fine but weaker JSON and constraints. MongoDB: weaker relational guarantees for ownership and auth
ORM / migrations	Prisma (or Drizzle) migrations in Git	Typed queries, reviewed SQL migrations	TypeORM
API style	**REST + OpenAPI 3.1**	See §11	GraphQL
Jobs	pg-boss (queue on PostgreSQL)	No extra infrastructure (no Redis needed at this scale)	BullMQ + Redis
Hosting	Docker Compose on one VM + managed PostgreSQL	Simple, cheap, scalable vertically and then horizontally	Kubernetes: not justified at this scale

5. Backend Architecture

```
backend/src
├── main.ts                    bootstrap: helmet, CORS allowlist, body limit, versioning /api/v1
├── config/                    env schema (zod) validated at boot; no defaults for secrets
├── common/
│   ├── filters/               ProblemDetailsFilter → RFC 9457 application/problem+json
│   ├── guards/                JwtAuthGuard, ChildOwnershipGuard, RateLimitGuard
│   ├── interceptors/          RequestId, Logging (redacted), Timeout
│   ├── pipes/                 ValidationPipe(whitelist, forbidNonWhitelisted, transform)
│   └── logger/                pino JSON; redact: password, token, authorization, cookie, email, name
├── modules/
│   ├── auth/      controller · AuthService · TokenService · PasswordService(Argon2id) · dto/
│   ├── users/     controller (/users/me) · UserService · UserRepository
│   ├── devices/   DeviceService (register, last_seen)
│   ├── children/  controller · ChildService · ChildRepository · dto/ (CreateChild, PatchChild)
│   ├── progress/  AttemptService · JourneyService · UsageService · repositories
│   ├── sync/      SyncController · SyncService(push/pull) · ConflictResolver · MutationLog
│   ├── email/     EmailService (provider adapter, templates, no PII in logs)
│   └── health/    /health/live, /health/ready (DB ping)
└── prisma/        schema.prisma · migrations/
```

Concern	Design
Controllers	Thin: parse DTO → call service → map to response DTO. Never return ORM entities.
Services	Business rules, transactions, ownership checks (defence in depth beyond guards)
Repositories	Every query takes `userId`; there are no "find by id" methods without an owner
DTOs / validators	class-validator or zod; strict types; length limits (name 1–24, Unicode letters/space/-/'); enum checks; max batch sizes
Auth	Passport-JWT (EdDSA or RS256 with `kid` rotation); opaque refresh tokens
Middleware	helmet, CORS (exact origins), compression, request-id, body limit 1 MB (sync), rate limits (per IP + per account)
Errors	Problem details with a stable `type` and `code`; no stack traces in production; generic auth errors
Logging	Structured JSON, correlation id, PII-redacted; audit events stored separately
Configuration	12-factor env vars; secrets from a secret manager or Docker secrets; fail fast if missing

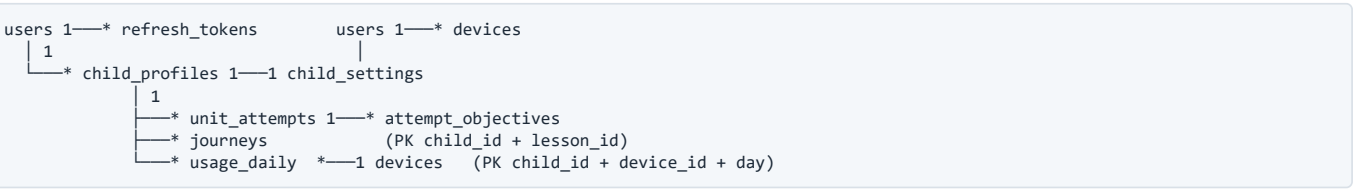
6. Database Architecture

Every primary key of a synced entity is a **client-generated UUIDv7**. Clients create records offline without collisions, IDs sort by time (index-friendly), and Postgres stores them natively. UUIDv7 was chosen over ULID for the native `uuid` type. Auto-increment IDs are not used for synced entities.

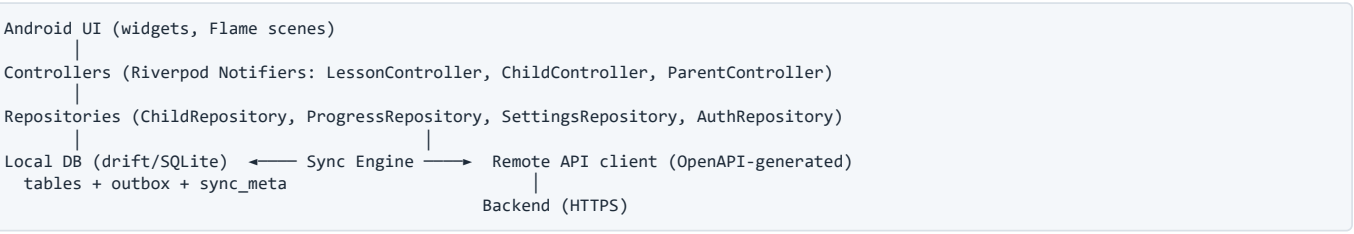
Table	Key columns	Constraints / indexes
users	id uuid PK, email citext, password_hash, email_verified_at, status, created_at, updated_at, deleted_at	UNIQUE(email) WHERE deleted_at IS NULL; CHECK status IN (…)
refresh_tokens	id, user_id FK, family_id, token_hash bytea, device_id, expires_at, revoked_at, replaced_by	UNIQUE(token_hash); INDEX(user_id), INDEX(family_id); ON DELETE CASCADE
email_tokens	id, user_id FK, purpose, token_hash, expires_at, used_at	UNIQUE(token_hash); single-use
devices	id uuid (client), user_id FK, platform, last_seen_at	INDEX(user_id)
child_profiles	id uuid (client), user_id FK, display_name, gender, avatar, palette, photo_object_key, version, change_seq, server_updated_at, deleted_at	INDEX(user_id, change_seq); CHECK length(display_name) BETWEEN 1 AND 24; max 10 children per user (service rule)
child_settings	child_id PK/FK, arabic, break_minutes, volume, sound, music, narration, boy_voice, girl_voice, version, change_seq	CHECK break_minutes BETWEEN 5 AND 30; CHECK volume BETWEEN 0 AND 1
unit_attempts	id uuid PK, child_id FK, unit_id, content_version, completed, effort_stars, completed_at, legacy_attempt_id, change_seq	INDEX(child_id, unit_id, completed_at DESC); CHECK effort_stars BETWEEN 0 AND 3; **append-only**
attempt_objectives	attempt_id FK, objective_id, outcome	PK(attempt_id, objective_id); CHECK outcome IN ('firstTry','afterHint','explained')
journeys	child_id FK, lesson_id, content_version, state jsonb, progress_rank, version, change_seq, deleted_at	PK(child_id, lesson_id); state validated against a JSON Schema per lesson version
usage_daily	child_id FK, device_id FK, day date, seconds	PK(child_id, device_id, day); CHECK seconds BETWEEN 0 AND 86400

processed_mutations	user_id, mutation_id, result jsonb, created_at	PK(user_id, mutation_id); TTL cleanup after 30 days
audit_log	id bigserial, user_id, action, meta jsonb, created_at	INDEX(user_id, created_at); no PII or tokens

Sync cursor. A global sequence `change_seq_seq`. Every insert, update or tombstone sets `change_seq = nextval()` inside the same transaction. Pull returns rows with `change_seq > :cursor` for the caller's children, ordered, with a page limit. (Commit-order edge cases are handled by reading only up to the minimum in-flight sequence, or by using a single writer transaction per push.)



7. Android Architecture



Decision	Category	Items
KEEP	Proven assets	Flutter + Riverpod; Flame scenes and art; content JSON + schemas + validators; `AttemptRecorder`, `addUsage`, unlock rules; photo sanitisation; MainActivity sound channel; manifest security posture; parent gate concept; existing tests
REFACTOR LATER	Structure	LessonPlayer → controller + scene renderers; `LessonSession` migrations → registry; `AppController` → repositories; error handling + logging; strings → ARB
REPLACE LATER	Foundations	Hive blob → drift tables + outbox; untyped journey maps → typed models; unbounded `Flame.images` → bounded loader; lossless assets → lossy WebP/AAB splits; parent gate → stronger variant
REMOVE LATER	Debt	`force_creature.dart`; 24 unused images from Git; preview routes in `main.dart` (move to `main_preview.dart`); `tmp/` patch-script workflow

Transition path (not executed): (1) Introduce repository interfaces backed by the existing Hive store, so behaviour does not change. (2) Add typed models and switch the controllers to the repositories. (3) Add the drift implementation plus a one-time local migration (§18), still offline. (4) Add the remote API and sync engine behind a feature flag, active only when a parent is logged in.

Android specifics: `INTERNET` permission (only after the privacy decision); `network_security_config` with cleartext disabled and system CAs only; WorkManager periodic sync (≥ 15 min, network-connected constraint); tokens in `flutter_secure_storage` (Keystore); flavours `dev / staging / prod` with `applicationIdSuffix` and `--dart-define-from-file` for `API_BASE_URL`.

8. Web Architecture

Concern	Design
Framework	Flutter Web (CanvasKit, already bundled locally); entry point `lib/main_web.dart`
Structure	Shared packages: `packages/domain` (models, merge rules), `packages/data` (repositories, sync), `packages/api_client` (generated), `packages/ui` (design system, scenes); thin app shells for Android and Web
Routing	`go_router` with URL paths: `/login`, `/children`, `/children/:id/reports`, `/play/:lessonId`, `/account`; auth redirect guard
Authentication	Refresh token in an `HttpOnly; Secure; SameSite=Strict` cookie scoped to `/api/v1/auth`; access token held in memory; silent refresh on load; CSRF: SameSite plus a required custom header on refresh
State	Riverpod (same as Android)
API layer	Generated Dart client with interceptors (auth, retry with backoff, problem+json mapping)
Local cache	drift on web (IndexedDB/OPFS) for offline viewing of non-sensitive progress; cleared on logout
Forms / validation	Shared validators from `domain` (same rules as the server DTOs); inline errors
Errors	Global error boundary; offline banner; accurate messages per problem `code`

Responsive	Breakpoints: <600 phone (single column), 600–1024 tablet (two-pane), >1024 desktop (sidebar + content). The game keeps a landscape aspect box with letterboxing.
Security headers	CSP (`script-src 'self' 'wasm-unsafe-eval'`), HSTS, X-Content-Type-Options, frame-ancestors 'none'

9. Authentication Architecture

Flow	Design
Register	Email plus password (≥ 12 chars, up to 128; breached-password check via k-anonymity range API; no composition rules). Same 202 response whether the email exists or not.
Email verification	256-bit random token, stored hashed (SHA-256), 24 h, single use; login allowed but sync blocked until verified
Login	Constant-time verification; generic error; rate limit 5/min per account and IP with progressive delay; device id recorded
Logout	Revoke the current refresh family; **logout-all** revokes all families
Forgot / reset password	Same generic response; token 30 min single use; on reset revoke all sessions and email a notification
Change password	Requires the current password; revokes other sessions
Account disable / delete	Soft delete with a 14-day grace period, then hard delete of all child data (right to erasure); export endpoint (JSON) before deletion
Future	Google / Microsoft sign-in via OIDC (Authorization Code + PKCE), linked to the same verified email

> **Password hashing: Argon2id** (OWASP Password Storage Cheat Sheet first choice). It is memory-hard, so it resists GPU and ASIC cracking better than bcrypt. bcrypt also truncates input at 72 bytes. Baseline parameters: m = 19 MiB, t = 2, p = 1, tuned so one hash takes about 100–300 ms on production hardware. Store the PHC string, which carries the parameters so they can be upgraded on next login. **bcrypt** (cost ≥ 12) is an acceptable fallback if the platform lacks Argon2.

Session security

Element	Design
Access token	JWT, 15 min, claims: `sub` , `sid` (family), `iat` , `exp` , `aud` , `iss` ; asymmetric signature with `kid` rotation; no PII
Refresh token	Opaque 256-bit random value; server stores only its hash; 30-day sliding window, 90-day absolute limit
Rotation	Each refresh issues a new token and marks the old one replaced
Reuse detection	Presenting an already-rotated token revokes the entire family (theft signal) and emails the parent
Revocation	Logout, password change or reset, account disable; access tokens expire within 15 min (optional deny-list by `sid` for immediate revocation)
Android storage	Refresh token in Keystore-backed `flutter_secure_storage` ; access token in memory; never Hive, logs or crash reports
Web storage	Refresh token in an HttpOnly Secure SameSite=Strict cookie; access token in memory. **No long-lived tokens in localStorage.**

10. Authorization Model

- **Tenant = parent account.** Every child, attempt, journey and usage row belongs to exactly one `user_id` (through `child_profiles.user_id`).
- The `user_id` always comes from the verified token `sub` and **never** from the path or body.
- `ChildOwnershipGuard` resolves `:childId` with `WHERE id = :childId AND user_id = :sub AND deleted_at IS NULL` . If nothing matches it returns **404**, not 403, so IDs cannot be enumerated.
- Sync push validates that every mutation's entity belongs to the caller inside the same transaction. Pull queries join through `child_profiles.user_id = :sub` .
- Optional defence in depth: PostgreSQL **Row-Level Security** with `SET app.user_id` per transaction.
- Automated BOLA/IDOR tests: user B's token against every user A resource ID must return 404.
- Roles for now: `parent` only. Future: `co_parent` (invited), `teacher` (read-only aggregated), modelled as an `account_members` table.

11. API Architecture

REST over GraphQL. Sync is batch push/pull, which maps naturally to two POST/GET endpoints. REST is easy to rate-limit, cache, log and secure per route. OpenAPI generates the Dart client for both apps. GraphQL's flexible querying adds complexity (query cost limits, authorization per field) without a benefit for this small, fixed data model.

Method	Endpoint	Purpose	Auth	Authorisation	Request → Response	Validation
POST	/api/v1/auth/register	Create account	None	—	{email, password} → 202	email RFC, pw 12–128, breached check, rate limit
POST	/api/v1/auth/verify-email	Verify email	None	token	{token} → 204	token format, unexpired, unused
POST	/api/v1/auth/resend-verification	Resend link	None	—	{email} → 202	rate limit
POST	/api/v1/auth/login	Start session	None	—	{email, password, deviceId, platform} → {accessToken, expiresIn, refreshToken*}	*body on Android, cookie on web; rate limit
POST	/api/v1/auth/refresh	Rotate tokens	Refresh token	family valid	{refreshToken} or cookie → new pair	reuse detection
POST	/api/v1/auth/logout	End session	Bearer	own session	{all?: bool} → 204	—
POST	/api/v1/auth/forgot-password	Send reset link	None	—	{email} → 202	generic response, rate limit
POST	/api/v1/auth/reset-password	Set new password	Reset token	token	{token, newPassword} → 204	pw policy; revoke sessions
POST	/api/v1/auth/change-password	Change password	Bearer	self	{currentPassword, newPassword} → 204	re-auth
GET	/api/v1/users/me	Profile	Bearer	self	→ {id, email, verified, createdAt}	—
DELETE	/api/v1/users/me	Delete account	Bearer + password	self	{password} → 202 (grace period)	re-auth
GET	/api/v1/users/me/export	Data export	Bearer	self	→ JSON of all data	rate limit
GET	/api/v1/children	List children	Bearer	owner	→ [ChildDto]	—
POST	/api/v1/children	Create child	Bearer	owner	{id(uuid), displayName, gender, avatar, palette} → 201 ChildDto	name 1–24 letters, enums, max 10 per user
PATCH	/api/v1/children/{childId}	Update child	Bearer	ChildOwnershipGuard	{...fields, baseVersion} + If-Match → 200 or 409 {server}	optimistic concurrency
DELETE	/api/v1/children/{childId}	Delete child	Bearer	ChildOwnershipGuard	→ 204 (tombstone)	—
GET / PUT	/api/v1/children/{childId}/settings	Settings	Bearer	ChildOwnershipGuard	SettingsDto + baseVersion	ranges
GET	/api/v1/children/{childId}/attempts?unitId=	Reports	Bearer	ChildOwnershipGuard	→ [AttemptDto] paginated	pagination limits
GET	/api/v1/children/{childId}/reports/summary	Parent report	Bearer	ChildOwnershipGuard	→ latest/best per unit, usage per day	—

POST	/api/v1/sync/push	Upload mutations	Bearer (verified)	each entity owned	{deviceId, mutations:[{mutationId, entity, op, id, baseVersion, payload}]} (≤100) → [{mutationId, status: applied|duplicate|conflict|rejected, version, changeSeq, server?, error?}]	per-entity schema; idempotency
GET	/api/v1/sync/pull?since=&limit=	Download changes	Bearer (verified)	joined by user_id	→ {changes:[…], cursor, hasMore}	limit ≤ 500
POST	/api/v1/devices	Register device	Bearer	self	{id, platform} → 204	uuid
GET	/health/live, /health/ready	Probes	None (internal)	—	→ 200	—

Conventions: JSON camelCase; RFC 3339 UTC timestamps; RFC 9457 errors; `Idempotency-Key` on non-sync POSTs; `/api/v1` URI versioning; cursor pagination.

12. Synchronization Architecture

```
Android UI → Controller → Repository ──(1 transaction)──► Local DB row (sync_status = pending, version++)
                                      └──────────────────► outbox(mutation_id UUIDv7, entity, op, base_version, payload)
Sync Engine (single-flight, triggered) ─► PUSH /sync/push (batches ≤ 100, ordered by outbox seq)
                                       ◄─ per-mutation results → mark synced / resolve conflict / quarantine
                                       ─► PULL /sync/pull?since=cursor (pages until hasMore = false)
                                       ◄─ upsert rows; rows with pending local edits go through merge
                                       → save new cursor (sync_meta) → UI streams update
```

Aspect	Design
Create	Client generates a UUIDv7; outbox op `create`; the server inserts, or returns `duplicate` if the id already exists for the same user with an identical payload
Update	Op `patch` with only the changed fields plus `baseVersion`; the server applies it if the version matches, otherwise runs the conflict resolver
Delete	Op `delete` → server tombstone (`deleted_at`, change_seq bump); tombstones kept ≥ 180 days so long-offline devices still receive them; afterwards a device with a stale cursor does a full resync
Initial sync	Upload linked local data (§18), then pull from cursor 0, paged
Incremental sync	Pull `since=cursor`; the server returns only changes for the caller's children
Automatic triggers	App start, app resume, 5 s debounce after a local change, connectivity regained, WorkManager every ≥ 15 min (Android), tab focus (web)
Manual	"Sync now" in the parent area, showing last sync time, pending count and errors
Retry	Exponential backoff 2^n s with full jitter, capped at 15 min; reset on success; network errors and 5xx/429 retried (honouring `Retry-After`)
Failed sync	A 4xx validation `rejected` moves the mutation to a **quarantine** table (never silently dropped) and shows it to the parent with options: retry after update, or discard (with confirmation)
Duplicate prevention	Client-generated IDs, plus `processed_mutations(user_id, mutation_id)` on the server (returns the stored result on replay), plus unique constraints
Ordering	The outbox is processed in order per entity; creates before patches; a parent (child profile) before dependants (attempts)
Concurrency	One sync at a time per device (mutex); push before pull

13. Offline Strategy

- **Offline is the default mode.** Every game feature works without a network or an account, exactly as today.
- The local DB is the source of truth for the UI. The server is the source of truth for cross-device convergence.
- Online mode only adds background push/pull. Nothing in the gameplay path ever waits on the network.
- When an access token expires offline, play continues; refresh happens on reconnect. If the refresh fails (revoked), local data is kept and the parent is asked to log in again; pending mutations wait.
- Logout asks whether to keep local children on the device (default: keep, unlinked) or remove them.

- Lesson content stays bundled. If a device is on an older `contentVersion`, server data from newer content is stored but rendered with the device's migration registry. If it is incompatible, the lesson shows "update the app to continue this lesson" while other data stays usable.

14. Conflict Resolution

Scenario: the parent edits the same record on the phone (offline) and on the computer. **Principle:** no blind last-write-wins for anything that represents learning evidence. Each entity type gets a merge rule that fits its meaning. Optimistic concurrency (`version`) detects the conflict, and `server_updated_at` and `device_id` serve as tie-breakers and audit.

Entity	Conflict type	Resolution	Data loss?
unit_attempts	None possible	**Append-only**, immutable, idempotent by id. Both devices' attempts are kept. "Latest" and "best" are computed.	No
attempt_objectives	None	Immutable child rows of an attempt	No
usage_daily	Concurrent increments	**Per-device counter** (PK includes device_id); totals = SUM over devices (a G-counter CRDT)	No
child_profiles / child_settings	Concurrent edits	**Field-level three-way merge** against the base version: disjoint fields are both applied automatically. The same field with different values → for settings (low value) the later `server receive time` wins and the overwritten value is recorded in audit; for profile identity fields (name, photo, gender) → the server keeps its value, returns a conflict, and the client shows the parent "Phone says X, Web says Y — choose".	No silent loss
journeys (in-progress lesson state)	Two devices advanced the same lesson	**Progress-monotonic merge:** union of completed step responses, keeping the **first recorded outcome** per objective (matches the existing `AttemptRecorder` rule against score inflation); position = the further one (`progress_rank`); the transient UI state of the losing side is discarded	No assessment loss
Delete vs edit	One side deleted a child, the other edited	**Delete wins** (privacy intent, confirmed behind the parent gate). The edit's attempts are quarantined and shown to the parent for 14 days before purge.	Recoverable for 14 days
Create duplicates	Same child created on two devices before linking	Never auto-merged by name. The parent sees both and may merge (attempts re-parented, journeys merged by the rule above).	No

Per-row metadata on the client: `version` (last server version seen), `sync_status` (synced | pending | conflict | quarantined), `updated_at` (local clock, informational only), `last_modified_device`. The server keeps `version`, `change_seq` and `server_updated_at`. Device clocks are **never** used to decide a winner.

15. Deployment Architecture

```
Users ──HTTPS──► Caddy (TLS auto-renew, HSTS, rate limit, gzip/brotli)
                  ├─ /           → web container (static Flutter Web build, long-cache hashed assets)
                  └─ /api/v1/*   → api containers (NestJS ×2, health-checked, rolling restart)
                                     ├─ PostgreSQL (managed, private network, TLS, PITR)
                                     ├─ Object storage (private bucket, signed URLs ≤ 5 min)
                                     └─ Email provider (SPF/DKIM/DMARC configured)
Ops: Uptime monitor · Prometheus/Grafana (or SaaS) · Loki logs · Sentry (PII scrubbing) · backups → off-site bucket
CI/CD: GitHub Actions → test → build images (pinned digests) → scan (Trivy/osv) → deploy staging → manual approval → prod
```

Docker: yes. Multi-stage images, non-root user, read-only filesystem, healthchecks, one Compose file per environment. **Kubernetes: not recommended.** The expected load (a family and education app, JSON sync) fits a single VM with vertical scaling. Revisit only if there are multiple regions, more than about 5 services, or sustained horizontal autoscaling needs. Android releases go through the Play Console (internal → closed → staged production rollout).

16. Backup Strategy

Item	Policy
Automatic backup	Managed PostgreSQL PITR, or self-hosted WAL-G/pgBackRest: continuous WAL archiving plus nightly base backup
Frequency	WAL continuous (target RPO ≤ 15 min); full base backup daily; logical `pg_dump` weekly (portability)

Retention	PITR window 14 days; daily backups 30 days; monthly backups 12 months (must not exceed the privacy policy's retention for deleted accounts — purge pipeline documented)
Security	Encrypted at rest (AES-256) and in transit; off-site bucket at a different provider or region; object lock / immutability against ransomware; separate credentials
Restore process	Runbook: provision a new instance → restore base + WAL to the target time → run migrations check → smoke tests → switch the connection string → verify sync cursors (clients with a cursor beyond the restore point do a full resync)
Restore testing	Monthly automated restore into staging plus integrity checks (row counts, checksum queries); quarterly timed drill
Disaster recovery	Target RTO ≤ 4 h (target, to be validated by drill); infrastructure as code (Compose + scripts) to rebuild in another region; DNS TTL 300 s
Client safety net	The devices' local DBs hold a full copy of their data and can re-push after a server restore (idempotent IDs)

17. Monitoring Strategy

Signal	Implementation	Alert
Application logs	pino JSON with request id, route, status, latency, user id hash; redaction list	Error-rate spike
Server logs	Caddy access logs (IP truncated), container stdout → Loki	—
API errors	5xx rate, problem-type counts; Sentry with `sendDefaultPii=false`	5xx > 1 % for 5 min
Database	Connections, slow queries (`pg_stat_statements`), replication/backup age, disk	Backup age > 26 h; disk > 80 %
Auth failures	Failed logins per account/IP, refresh-token reuse events, reset requests	Reuse detected; brute-force pattern
Sync health	Push/pull latency, conflict rate, quarantine count, client-reported lag	Conflict or quarantine spike
Performance	p50/p95/p99 latency per route; CPU/RAM	p95 > 500 ms for 10 min
Uptime	External probe on `/health/ready` and the web root every minute	2 consecutive failures
Audit logs	Login, logout, password change/reset, email change, child create/delete, account delete, data export	—
Mobile	Opt-in, privacy-safe crash reporting (no names or photos); local diagnostic log exportable by the parent	Crash-free sessions < 99.5 %

Never logged: passwords, tokens (access, refresh, reset, verify), Authorization and Cookie headers, child names, photos, emails in plaintext (hash them when correlation is needed).

18. Migration Strategy (existing local data, zero data loss)

```
Existing Hive blob (schemaVersion 3)
  ↓ 1. Local upgrade (offline): parse per child, tolerant; a bad child is isolated, not fatal
  ↓ 2. Write all rows to SQLite in ONE transaction; keep the Hive blob untouched as a backup (≥ 2 releases)
  ↓ 3. Verify counts and checksums (children, attempts, objectives, journeys, usage-days) → else roll back and keep Hive
  ↓ 4. ID normalisation: 'local-child' and non-UUID ids → UUIDv7 (legacy_id kept);
  ↓    attempt ids '{lesson}-{micros}' → UUIDv7 with legacy_attempt_id
  ↓ 5. Parent creates an account or logs in (behind the parent gate)
  ↓ 6. Parent confirms "Link these children to my account" (per child; unlinked children stay local)
  ↓ 7. Initial upload via /sync/push in batches (idempotent mutation ids; resumable after interruption)
  ↓ 8. Server validation: applied → synced; rejected → quarantined locally (shown, never deleted)
  ↓ 9. Pull from cursor 0 (brings children created on other devices)
  ↓ 10. If both sides have children: show both, parent decides merge or keep (no auto-merge)
  ↓ 11. Enable incremental sync
```

Safeguards: a migration dry-run mode in debug builds; a test matrix of real anonymised blobs from each historic schema (v1, v2, v3 plus each journey version); telemetry-free success flag stored locally; a "rescue export" button in the parent area that writes the raw local data to a user-chosen file.

19. Development Roadmap

Phase	Name	Priority	Depends on	Complexity	Key deliverables

0	Preparation, CI, reproducible build, signing key	P0	—	M	Green CI, release AAB, device test report
1	Fix critical existing issues + privacy decision	P0/P1	0	M	Patch release, decision record
2	Android refactor: typed models, repositories, drift, local migration	P0	1	XL	Behaviour-identical refactored app
3	Database design and API contract (OpenAPI, sync spec, threat model)	P1	2	M	openapi.yaml, ERD, sync spec
4	Backend core + authentication	P1	3	L	API service, auth tests
5	Android API integration (optional accounts)	P1	4	M	App with optional login
6	Synchronisation	P1	5	XL	Multi-device sync, test report
7	Web application (Flutter Web)	P2	6	M–L	Web app on staging
8	Security hardening + pentest	P1	4–7	M	Pentest report, fixes
9	Testing and QA (device farm, a11y, load)	P1	7	M–L	QA sign-off
10	Staging	P1	9	S–M	RC builds, closed testing
11	Production (staged rollout)	P1	10	M	Live system, dashboards, runbooks

No calendar durations are given. Team size, availability and portal scope are unknown, so week estimates would be false precision. Re-estimate after Phase 0. Full per-phase objectives, tasks, risks and skills: `Development_Roadmap.md`.

20. Risks

Risk	Likelihood	Impact	Mitigation
Children's-privacy and legal non-compliance	Medium	Very high	Legal review in Phase 1; parent-owned accounts; consent; minimisation; no photo sync by default; export and deletion
Data loss during local migration	Low–Medium	Very high	Transactional migration, checksums, keep the Hive backup, rescue export, test matrix
Sync bugs (lost or duplicated progress)	Medium	High	Per-entity merge rules, idempotency, property-based and two-device tests, quarantine instead of delete
Loss of the "zero permissions / offline" trust	Medium	Medium	Accounts optional; clear explanation; no analytics
Signing key loss	Low	Very high	Play App Signing, two offline backups, documented custody
Refactor regressions in scenes	Medium	Medium	Existing 203 tests plus goldens; behaviour-identical refactor release before any backend work
Flutter Web performance (bundle, first load)	Medium	Medium	Deferred imports, asset optimisation, caching; Next.js only for a public landing site
Small team / single maintainer	High	High	CI, docs, reproducible toolchain, a managed DB to reduce operations load
Account takeover	Low	High	Argon2id, rate limits, token rotation with reuse detection, notifications, optional 2FA later

21. Final Recommendation

> **B — Refactor the existing application.** Keep the Flutter client (Android + Web from one codebase). Rebuild the data layer (typed models, repositories, drift/SQLite, outbox). Add a **NestJS + PostgreSQL** backend with a **REST/OpenAPI** contract, **parent email authentication** (Argon2id, 15-min JWT + rotating refresh tokens with reuse detection, Keystore / HttpOnly cookies) and **offline-first sync** (push/pull with change_seq cursors, idempotent mutations, optimistic concurrency, per-entity merge rules that never silently lose learning data).
>
> **Precondition:** revise the "no accounts / no data sending" decision with privacy and legal input before Phase 3.

Appendix A: Environments

	Development	Staging	Production
Backend	Docker Compose locally, hot reload	Same images as prod, separate VM	Hardened VM(s)
Database	Local Postgres container, seeded **synthetic** data	Separate managed instance; synthetic data only (never real child data)	Managed PostgreSQL with PITR
API URL	`http://localhost:3000` (emulator `10.0.2.2`)	`https://api.staging.<domain>`	`https://api.<domain>`
Android	flavour `dev`, id suffix `.dev`; cleartext allowed for localhost only (dev network config)	flavour `staging`, `.staging`; HTTPS only	flavour `prod`; HTTPS only; obfuscated
Config	`.env.example` committed, `.env` ignored; `--dart-define-from-file=env/dev.json`	Secrets in CI secret store / secret manager	Secret manager; rotated JWT keys (`kid`)
Email	Local catcher (Mailpit)	Provider sandbox	Provider with SPF/DKIM/DMARC

Rules: secrets never in Git (secret scanning in CI); each environment has its own DB credentials and JWT keys; configuration validated at boot; client builds embed only public configuration (API base URL), never secrets.

Appendix B: Future Repository / Project Structure (proposal, not created)

```
/science-adventure
├── apps/
│   ├── mobile/            Flutter app shell (Android) — flavours dev/staging/prod
│   └── web/               Flutter Web shell (main_web.dart, web/index.html, CSP)
├── packages/
│   ├── domain/            typed models, merge rules, validators (pure Dart, 100 % unit tested)
│   ├── data/              repositories, drift DB, outbox, sync engine
│   ├── api_client/        generated from openapi.yaml
│   ├── ui/                design system, scenes, Flame games, assets
│   └── content/           lesson JSON, schemas, migration registry
├── backend/               NestJS service (src/, prisma/, test/)
├── database/              ERD, migration notes, seed scripts (synthetic only)
├── api/                   openapi.yaml (single source of truth)
├── infrastructure/        docker-compose.*.yml, Caddyfile, backup scripts, runbooks
├── tools/content/         Python content pipeline + validators
├── docs/                  ADRs, privacy, threat model, runbooks
└── .github/workflows/     ci-mobile.yml, ci-backend.yml, deploy.yml, security-scan.yml
```

A monorepo (melos for Dart packages) is recommended so the API contract, the Dart client and the server stay in one change set.